

İNÖNÜ ÜNİVERSİTESİ

**MÜHENDİSLİK FAKÜLTESİ
YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ**

**B ve B+ Ağaçlarının Günümüzde Kullanım Alanları ve Yapısal
Analizi**

Veri Yapıları Dersi Araştırma Ödevi

Hazırlayan: Ahsenur Merve Balta

Öğrenci No: 02240224093

1. Giriş

Modern bilgisayar sistemlerinde ve yazılım mimarilerinde, verinin kalıcı depolama birimlerinde (HDD, SSD) nasıl organize edildiği sistem performansını doğrudan etkileyen en temel unsurdur. İşlemci (CPU) ve rastgele erişimli bellek (RAM) hızları mikrosaniyeler ve nanosaniyeler mertebesinde çalışırken, ikincil depolama birimlerine erişim (Disk I/O) milisaniyeler seviyesinde kalarak sistemlerde ciddi darboğazlar (bottleneck) oluşturmaktadır. Bu hız uçurumunu kapatmak ve disk üzerindeki blok tabanlı okuma/yazma süreçlerini optimize etmek amacıyla, dengeli arama ağaçları olan B-Ağacı (B-Tree) ve B+-Ağacı (B+ Tree) veri yapıları geliştirilmiştir. Bu çalışma kapsamında, söz konusu veri yapılarının günümüz modern sistemlerdeki kullanım alanları, avantajları ve gelecekteki konumları analiz edilmiştir.

2. B ve B+ Ağaçlarının Günümüzde Kullanım Alanları

2.1. Veri Tabanı Yönetim Sistemlerinde (DBMS) Kullanımı

İlişkisel ve ilişkisel olmayan veri tabanlarında sorgu optimizasyonu, minimum disk I/O işlemi yapılmasına bağlıdır. B ve B+ ağaçları, disk bloklarının fiziksel yapısıyla (genellikle 4KB veya 16KB'lık sayfalar) birebir uyumlu çalışacak şekilde geniş ve yatay bir hiyerarşide kurulur.

- **MySQL (InnoDB):** MySQL'in varsayılan depolama motoru olan InnoDB, tabloların birincil anahtarlarını (Primary Key) ve verinin kendisini kümelenmiş indeks (Clustered Index) mimarisinde saklamak için **B+ Tree** yapısını kullanır. Veriler doğrudan en alttaki yaprak düğümlerde sıralı olarak tutulur.
- **PostgreSQL:** PostgreSQL, standart indeksleme protokolü olarak yüksek verimli bir **B-Tree** varyasyonu kullanır. PostgreSQL mimarisindeki bu yapı, aralık sorgularını hızlandırmak adına yaprak düğümleri birbirine bağlayan çift yönlü bir liste içerir ve bu yönüyle B+ yapısının avantajlarını da barındırır.
- **MongoDB:** Doküman tabanlı popüler bir NoSQL veri tabanı olan MongoDB, varsayılan depolama motoru WiredTiger üzerinde indeksleme yaparken **B-Tree** yapısından yararlanır. Bu tercih, milyarlarca doküman arasından belirli bir anahtara (ID) göre tekil aramalarda doğrudan düğüm içinden veriye ulaşma hızı sağlar.

2.2. İşletim Sistemleri ve Dosya Sistemlerinde Kullanımı

Dosya sistemleri, depolama aygıtındaki milyonlarca dosyanın metadata bilgilerini (ad, boyut, oluşturulma tarihi, fiziksel konum vb.) yönetmek için bu ağaç yapılarından faydalanır.

- **NTFS (Windows):** Klasör dizinlerini ve dosya indekslerini hızlı bir şekilde eşleştirmek amacıyla B+ Tree mimarisini kullanır. Bir dizin içerisindeki binlerce alt dosya arasında arama yapıldığında, disk kafasının doğrusal arama yapmak yerine logaritmik adımlarla hedefe ulaşmasını sağlar.
- **EXT4 (Linux):** Linux ekosisteminin yaygın dosya sistemi EXT4, çok büyük dizin mimarilerini yönetebilmek adına **Htree** adı verilen, B-Tree tabanlı özel bir karma (hashing) ve ağaç yapısı kullanır.
- **APFS (Apple File System):** macOS, iOS ve iPadOS cihazlarda kullanılan modern APFS, dinamik disk alanı tahsisi ve dosya sisteminin meta verilerinin hızlı taranması için B+ Tree türevlerini temel alır.

2.3. Arama Motorları ve Büyük Veri Sistemleri

Büyük veri çağında verinin çok hızlı yazılması istendiğinde genellikle LSM Tree (Log-Structured Merge-tree) mimarileri tercih edilse de, verilerin disk üzerinde indekslenmesi ve hızlı sorgulanması aşamalarında B ve B+ ağacı mantığı önemini korumaktadır. Google ve Elasticsearch gibi sistemlerin temelini oluşturan "Ters Dizin" (Inverted Index) yapılarında, aranan terimlerin sözlük (dictionary) indeksleri arka planda B-Tree benzeri hiyerarşik yapılarla saklanarak milisaniyeler içinde döküman eşleşmesi gerçekleştirilir.

2.4. Modern Sistemlerde Tercih Edilme Nedenleri

B ve B+ ağaçlarının klasik ikili arama ağaçlarına (BST) veya AVL ağaçlarına göre modern sistemlerde standart olarak tercih edilmesinin temel nedenleri şunlardır:

- 1. Yüksek Dallanma Katsayısı (High Fan-out) ve Düşük Yükseklik:** İkili ağaçlarda her düğümün en fazla iki çocuğu olabilirken, B/B+ ağaçlarında bir düğümde yüzlerce veya binlerce çocuk düğüm adresi tutulabilir. Bu sayede milyonlarca veya milyarlarca satırlık bir veride bile ağacın yüksekliği (kökten yaprağa olan mesafe) genellikle 3 ya da 4 katı geçmez. Bu da aranan veriye ulaşmak için en fazla 3-4 disk okuması (I/O) yapılacağı anlamına gelir.
- 2. Disk Sayfa (Page/Block) Uyumu:** İşletim sistemleri diskten veriyi byte bazında değil, bloklar (sayfalar) halinde okur. B/B+ ağaçlarının bir düğüm boyutu, diskin bir sayfa boyutuna (örneğin 16KB) denk gelecek şekilde ayarlanır. Böylece tek bir disk erişiminde yüzlerce indeks veya veri elemanı aynı anda RAM belleğe yüklenir.
- 3. B+ Tree'nin Sıralı Erişim ve Aralık Sorgusu Avantajı:** B+ Tree yapısında tüm gerçek veriler yalnızca en alt katmandaki yaprak düğümlerde (Leaf Nodes) saklanır ve bu yapraklar birbirine bağlı bir liste (Linked List) mimarisi ile bağlanmıştır. Bu sayede, belirli bir aralıktaki veriler sorgulandığında (örneğin belirli tarihler arasındaki işlemler), ağaç kökten aşağıya tekrar tekrar taranmaz. İlk eleman bulunduktan sonra yapraklar üzerinde sağa doğru doğrusal olarak ilerlenerek veriler çok hızlı bir şekilde okunur.

3. Günlük Hayattan Bir Benzetme: Hastane Arşiv Sistemi

B-Tree ve B+ Tree yapılarının mantığını, içerisinde milyonlarca hastaya ait fiziksel dosyaların bulunduğu büyük bir **Hastane Arşiv Sistemi** benzetmesiyle somutlaştırabiliriz.

3.1. B-Tree Modeli ile Arşiv Yönetimi

Bu senaryoda, arşiv odasında ana yönlendirici dolaplar, alt çekmeceler ve içlerinde hasta dosyaları bulunmaktadır. Arşiv görevlisi odaya girdiğinde ilk büyük dolaba (Kök Düğüm) bakar. Dolabın üzerinde "A-K arası alt çekmcelere, L-Z arası yan dolaba" gibi yönlendirmeler vardır. Ancak B-Tree modelinde, bu yönlendirici dolapların ve rafların üzerinde ****bazı hastaların doğrudan orijinal dosyaları da**** yer almaktadır. Eğer görevlinin aradığı hasta, şans eseri o üst raflardaki popüler veya sık gelen bir hastayla, görevli en alt çekmcelere inmeden dosyayı hemen oradan alır ve işlem biter. Eğer aranan hasta orada değilse, yönlendirmeleri takip ederek alt çekmcelere doğru iner.

Faydası: Sık aranan veya popüler verilere ağacın üst katmanlarında denk gelindiğinde, arama süreci çok daha kısa sürer ve en alta inme ihtiyacını ortadan kaldırır.

3.2. B+ Tree Modeli ile Arşiv Yönetimi (Modern Sistemlerin Tercihi)

Bu modelde ise sistem çok daha disiplinli ve optimize edilmiştir. Üst katmanlardaki ana dolaplarda ve raflarda ****asla gerçek hasta dosyası bulunmaz****. Buralarda sadece ve sadece yönlendirici etiketler, harf aralıklarını gösteren levhalar (indeksler) yer alır. Tüm hastaların gerçek fiziksel dosyaları, ****en alt kattaki çekmecelerde (Yaprak Düğüm)** alfabetik ve kronolojik olarak kusursuz bir sıra ile ****dizilmiştir**. Ayrıca, en alt kattaki her bir çekmecenin arka kısmından bir sonraki çekmeceye doğrudan açılan gizli bir geçit (Linked List bağı) bulunmaktadır.

Görevliye ****Soyadı B ile başlayan ve bugün muayene olacak tüm hastaların dosyalarını getir**** şeklinde bir aralık veya grup sorgusu talimatı verildiğinde; görevli üst levhalara bakarak hızlıca aşağı iner ve en alt kattaki B harfinin başladığı ilk çekmeceyi bulur. Dosyayı aldıktan sonra tekrar yukarı katlara çıkıp yönlendirmelere bakmasına gerek kalmaz; çekmecelerin arkasındaki gizli geçitleri kullanarak sırayla sağa doğru ilerler ve tüm B harfli dosyaları kesintisiz bir şekilde toplar.

Somut Faydası ve Avantajları:

- **Öngörülebilir ve Tutarlı Performans:** Her hastanın dosyası istisnasız en alt katta yer aldığı için, arama adımı her sorgu için sabittir ve tutarlıdır.
- **Etkin Aralık Sorguları:** Üst raflar dosya kalabalığından arındırıldığı ve sadece yönlendirme levhaları tutulduğu için, tek bir rafa çok daha fazla yönlendirme sığdırılabilir (Yüksek dallanma katsayısı). Yapraklar arasındaki bağ sayesinde sıralı veri tarama işlemleri maksimum hızda gerçekleşir.

4. Yapısal Değerlendirme ve Analiz (Kişisel Yorumlar)

4.1. Bu Veri Yapıları Neden Önemlidir?

Kişisel Yorum: Bilgisayar donanımları incelendiğinde işlemci hızlarının muazzam bir boyuta ulaştığı, ancak kalıcı depolama birimlerinin okuma/yazma hızlarının bu ivmeye yetişemediği görülmektedir. Yazılım mühendisliğinde performans optimizasyonunun kalbi, diske erişim sayısını minimumda tutmaktır. B ve B+ ağaçları, RAM ile kalıcı disk arasındaki bu fiziksel hız uçurumunu kapatan matematiksel bir köprüdür. Diskin donanımsal çalışma mantığı olan blok okuma sistemini, yazılımsal bir algoritma ile mükemmel şekilde eşleştirirler. Bu yapılar olmasaydı, günümüz dünyasındaki dijital sistemlerin tamamı I/O kuyruklarında kilitlenmeye mahkum olurdu.

4.2. Büyük Veri Çağında Neden İhtiyaç Duyulmaktadır?

Kişisel Yorum: Büyük veri (Big Data) kavramı sadece verinin boyutuyla değil, o veriye anlık olarak erişim ihtiyacıyla da doğrudan ilişkilidir. Milyarlarca satırdan oluşan veri setlerinde arama yaparken diski doğrusal olarak taramak (Full Table Scan) imkansızdır. Büyük veri çağında bir algoritmanın başarısı, aranan verinin nerede olduğunu bulmaktan ziyade, "nerede olmadığını" ilk saniyede elleyebilme yeteneğidir. B ve B+ ağaçları, veri boyutu Petabayt seviyelerine çıksa bile arama karmaşıklığını $O(\log n)$ seviyesinde sabit tutarak, büyümenin doğrusal değil logaritmik olmasını sağlar. Bu sayede sistemlerin ölçeklenebilir ve sürdürülebilir kalmasına imkan tanır.

4.3. Gelecekte de Kullanılmaya Devam Edecek midir?

Kişisel Yorum: Kısa ve orta vadede bu yapıların tahtından indirilmesi mümkün görünmemektedir; ancak yapısal evrimler geçirmektedirler. NVMe SSD'lerin ve tamamen RAM tabanlı (In-Memory) veri tabanlarının yaygınlaşması, mekanik disk kısıtlamalarını büyük ölçüde esnetmiştir. Tamamen bellek üzerinde çalışan sistemlerde B-Tree yerine SkipList veya Radix Tree gibi yapılar ön plana çıkabilmektedir.

Ayrıca günümüzde yapay zeka destekli "**Öğrenilmiş İndeksler**" (**Learned Indexes**) mimarileri üzerine yoğun akademik çalışmalar yürütülmektedir (Google'ın veri dağılımını makine öğrenmesi modelleriyle tahmin etmesi gibi). Bu yeni yaklaşımlar, geleneksel B-Tree yapılarına kıyasla %70'e varan bellek tasarrufu ve daha yüksek hızlar vaat etmektedir.

Buna rağmen B ve B+ ağaçlarının deterministik (kesin ve öngörülebilir) yapısı, eşzamanlılık (concurrency) ve kilit yönetimi (locking) konularındaki endüstriyel olgunluğu, onları daha uzun yıllar boyunca ilişkisel veri tabanı motorlarının vazgeçilmez temel taşı yapmaya devam edecektir. Gelecek teknolojiler onu yok etmek yerine, muhtemelen onun üzerine hibrit modeller inşa edecektir.